



**TECNOLÓGICO
NACIONAL DE MÉXICO**
INSTITUTO TECNOLÓGICO DE TIJUANA



INSTITUTO TECNOLÓGICO DE TIJUANA
DEPARTAMENTO DE SISTEMAS COMPUTACIONALES

SEMESTRE

Enero – Junio 2026

CARRERA

Ingeniería en Sistemas Computacionales

MATERIA

Patrones de Diseño

TÍTULO

Patrón de diseño Estado

NOMBRE Y NO. DE CONTROL DEL ALUMNO

Gayosso Pérez Karla Yuray – 22210764

NOMBRE DEL DOCENTE

Maribel Guerrero Luis

¿Qué es el patrón de diseño de estado?

Este es un patrón que permite que un objeto cambie su forma de actuar cuando cambia su estado interno, es decir, el objeto se comporta diferente dependiendo en qué estado se encuentre. Este sustituye los varios “if” o “switch” para manejar comportamientos, es decir que:

- Encapsula cada estado en una clase diferente
- Permite cambiar de estado dinámicamente
- Hace un código más limpio

Ventajas:

- Evita muchos “if” y “switch”
- Código más organizado
- Se pueden agregar nuevos estados fácilmente
- Cumple el principio abierto/cerrado

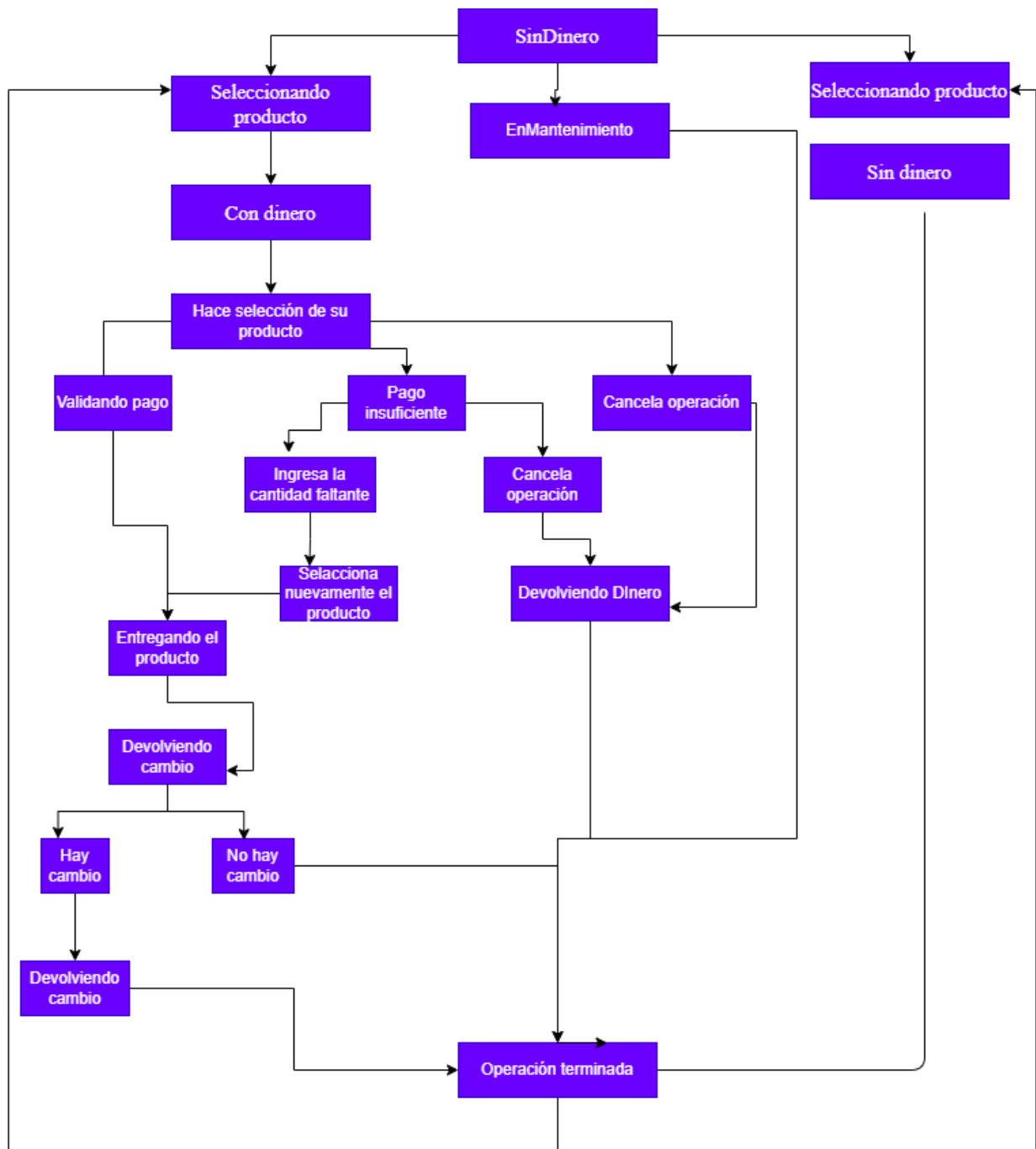
Puede agregar nuevos estados sin modificar el código existente, solo creando nuevas clases

- Facilita el mantenimiento

Desventajas:

- Puede generar muchas clases
- Aumenta la complejidad si el sistema es simple

Diagrama de estados



Capturas del programa

Idle
IdleState

Esperando selección

> Selecciona un producto para comenzar.

Insertado
\$0 MXN

PRODUCTOS



Coca-Cola
\$18 MXN
Stock: 3



Papas Lays
\$15 MXN
Stock: 2



Agua
\$12 MXN
¡Último!



Snickers
\$22 MXN
Stock: 5



Jugo Naranja
\$20 MXN
Stock: 2



Café
\$25 MXN
¡Último!



Galletas Oreo
\$14 MXN
Stock: 4



Gatorade
\$28 MXN
Stock: 2

INSERTAR MONEDA

\$5

\$10

\$20

\$50

\$100

\$ Otra cantidad

Insertar

Mantenimiento

Seleccionado
ItemSelectedState

Inserta dinero para continuar

> ☒ Seleccionaste: Coca-Cola — \$18.00

Insertado
\$0 MXN

Precio
\$18 MXN

PRODUCTOS



Coca-Cola
\$18 MXN
Stock: 3



Papas Lays
\$15 MXN
Stock: 2



Agua
\$12 MXN
¡Último!



Snickers
\$22 MXN
Stock: 5



Jugo Naranja
\$20 MXN
Stock: 2



Café
\$25 MXN
¡Último!



Galletas Oreo
\$14 MXN
Stock: 4



Gatorade
\$28 MXN
Stock: 2

INSERTAR MONEDA

\$5

\$10

\$20

\$50

\$100

\$ Otra cantidad

Insertar

MXN · Monedas desde \$1 · Billetes hasta \$500

✕ Cancelar

>  \$20.00 insertado – Total: \$20.00 / \$18.00

Insertado
\$20 MXN

Precio
\$18 MXN

PRODUCTOS

 Coca-Cola \$18 MXN Stock: 3	 Papas Lays \$15 MXN Stock: 2	 Agua \$12 MXN ¡Último!	 Snickers \$22 MXN Stock: 5	 Jugo Naranja \$20 MXN Stock: 2	 Café \$25 MXN ¡Último!	 Galletas Oreo \$14 MXN Stock: 4	 Gatorade \$28 MXN Stock: 2
--	---	---	---	---	---	--	---

INSERTAR MONEDA

\$5	\$10	\$20	\$50	\$100
-----	------	------	------	-------

\$ Otra cantidad

Insertar

✓ Confirmar

✕ Cancelar

DIAGRAMA DE ESTADOS



Insertado
\$0 MXN

PRODUCTOS

 Coca-Cola \$18 MXN Stock: 2	 Papas Lays \$15 MXN Stock: 2	 Agua \$12 MXN ¡Último!	 Snickers \$22 MXN Stock: 5	 Jugo Naranja \$20 MXN Stock: 2	 Café \$25 MXN ¡Último!	 Galletas Oreo \$14 MXN Stock: 4	 Gatorade \$28 MXN Stock: 2
--	---	---	---	---	---	--	---

INSERTAR MONEDA

\$5	\$10	\$20	\$50	\$100
-----	------	------	------	-------

\$ Otra cantidad

Insertar

 Mantenimiento

DIAGRAMA DE ESTADOS



ÚLTIMAS VENTAS

 Coca-Cola \$18 MXN cambio \$2 MXN

PRODUCTOS



Coca-Cola
\$18 MXN
Stock: 2



Papas Lays
\$15 MXN
Stock: 2



Agua
\$12 MXN
¡Último!



Snickers
\$22 MXN
Stock: 4



Jugo Naranja
\$20 MXN
Stock: 2



Café
\$25 MXN
¡Último!



Galletas Oreo
\$14 MXN
Stock: 4



Gatorade
\$28 MXN
Stock: 2

INSERTAR MONEDA

Insertar

✓ Confirmar

✕ Cancelar

DIAGRAMA DE ESTADOS



ÚLTIMAS VENTAS

 Snickers	\$22 MXN cambio \$78 MXN
 Coca-Cola	\$18 MXN cambio \$2 MXN

Funcionamiento

Patrón de diseño Estado, este mismo resuelve un problema específico (la máquina expendedora) la cual se comporta dependiendo la acción “actual” en la que se encuentra.

Dentro de mi proyecto se encuentran 3 puntos importantes:

- Clase abstracta

Define nuestro contrato

- Estados concretos

Implementa ese contrato cada uno a su manera

- Contexto

La máquina expendedora asigna todo al estado activo

La clase base VendingMachine (Máquina expendedora)

Define todos los métodos posibles de la máquina, es decir, cada método llama a `_invalid()`, la cual muestra un aviso en el log.



```
35     this._cb.setProducts([...this._products]);
36     this._cb.setSales([]);
37 }
38
39
40 selectItem(item) { this._state.selectItem(item); }
41 insertMoney(amount) { this._state.insertMoney(amount); }
42 confirmPurchase() { this._state.confirmPurchase(); }
43 cancel() { this._state.cancel(); }
44 requestMaintenance(){ this._state.requestMaintenance(); }
45 endMaintenance() { this._state.endMaintenance(); }
46 refund() { this._state.refund(); }
47
48 restock(productId, quantity) {
49     if (this._state.restock) {
50         this._state.restock(productId, quantity);
51     } else {
52         this.setLog('⚠ Solo se puede reabastecer en modo mantenimiento.');
```

Programa principal

State Maquina ⓘ
All changes saved 5 days ago. ✓

Expo Docs

Open files

Project

- context
 - VendingMachine.js
- data
- states
- styles
- ui
- App.js
- package.json
- README.md

```
1 import { IdleState } from '../states';
2
3 export default class VendingMachine {
4   constructor(callbacks) {
5     this._cb = {
6       setStateName : callbacks.setStateName,
7       setStateDesc : callbacks.setStateDesc,
8       setLog       : callbacks.setLog,
9       setInserted  : callbacks.setInserted,
10      setSelected   : callbacks.setSelected,
11      setChangeAmt  : callbacks.setChangeAmt,
12      setProducts   : callbacks.setProducts,
13      setSales      : callbacks.setSales,
14    };
15
16    this._insertedMoney = 0;
17    this._selectedItem  = null;
18    this._change        = 0;
19    this._availableChange = 200;
20    this._sales          = [];
21
22    this._products = [
23      { id: 1, name: 'Coca-Cola', price: 18, emoji: '🥤', stock: 3 },
24      { id: 2, name: 'Papas Lays', price: 15, emoji: '🍟', stock: 2 },
25      { id: 3, name: 'Agua', price: 12, emoji: '💧', stock: 1 }.
```

Configuración de la UI por estado

State Maquina ⓘ
All changes saved 5 days ago. ✓

Open files

Project

- context
- data
- stateMeta.js
- states
- styles
- ui
- App.js
- package.json
- README.md

```
1 export const STATE_META = {
2   IdleState: {
3     label   : 'Idle',
4     desc    : 'Esperando selección',
5     color   : '#6366f1',
6     bg      : '#eef2ff',
7     icon    : '🟦',
8     actions : ['select', 'maintenance'],
9   },
10  LowStockState: {
11    label   : 'Stock bajo',
12    desc    : 'Último artículo',
13    color   : '#f59e0b',
14    bg      : '#fffbeb',
15    icon    : '🟡',
16    actions : ['insertMoney', 'cancel'],
17  },
18  ItemSelectedState: {
19    label   : 'Seleccionado',
20    desc    : 'Inserta dinero',
21    color   : '#f59e0b',
22    bg      : '#fffbeb',
23    icon    : '🟡',
24    actions : ['insertMoney', 'cancel'],
25  },
26 }
```

La pantalla (vendingScreen)

 **State Maquina** ⓘ
All changes saved 5 days ago. ✓

Expo Docs

Open files

Project

ui

components

ActionButtons.js

CoinPanel.js

Log.js

MoneyDisplay.js

ProductGrid.js

SalesLog.js

StateBar.js

StateDiagram.js

screens

VendingScreen.js

```
1 import React, { useState, useEffect, useRef } from 'react';
2 import { View, Text, ScrollView } from 'react-native';
3
4 import VendingMachine from '../context/VendingMachine';
5 import { STATE_META } from '../data/stateMeta';
6 import { global } from '../styles/globalStyles';
7
8 import StateBar from '../components/StateBar';
9 import LogBox from '../components/Log';
10 import MoneyDisplay from '../components/MoneyDisplay';
11 import ProductGrid from '../components/ProductGrid';
12 import CoinPanel from '../components/CoinPanel';
13 import ActionButtons from '../components/ActionButtons';
14 import StateDiagram from '../components/StateDiagram';
15 import SalesLog from '../components/SalesLog';
16
17 export default function VendingScreen() {
18   const [stateName, setStateName] = useState('IdleState');
19   const [stateDesc, setStateDesc] = useState('Esperando selección');
20   const [log, setLog] = useState('Selecciona un producto para
comenzar.');
```

Interfaz

ui

components

ActionButtons.js

CoinPanel.js

Log.js

MoneyDisplay.js

ProductGrid.js

SalesLog.js

StateBar.js

StateDiagram.js

```
11
12 export default function ActionButtons({ stateName, onConfirm, onCancel,
onMaintenance, onEndMaintenance }) {
13   const meta = STATE_META[stateName] || STATE_META.IdleState;
14   const actions = meta.actions;
15
16   const handlers = {
17     confirm : onConfirm,
18     cancel : onCancel,
19     maintenance : onMaintenance,
20     endMaintenance: onEndMaintenance,
21   };
22
23   if (actions.length === 0) return null;
24
```

Conclusión

En resumen, este patrón es fácil de utilizar si va enfocado a estados pequeños, es decir, que dentro de un proyecto mayor podría volverse tedioso de acuerdo a las separaciones que se hacen por este patrón. La máquina expendedora aunque sea leve y ligero se lleva un diagrama más extenso porque debemos recordar en poner estado por estado